

LOAD-TEST SWEEP · THREE PROFILES

# Spike Sweep a05409f

Three k6 spike runs on one commit. The system passed everything that matters; the report generator misread all three.

2026-08-18 · 19:09 → 19:52 UTC

darwin/arm64 · 11 CPU · k6 over SSH

2.1 M tickets sold · 0 5xx · 0 failed orders

## These are not three attempts at one test

READ FIRST

Same commit, same host, three different `LOAD_PROFILE` values. They answer three separate questions and their throughput numbers are not comparable to each other:

- **19:09** — `browse-and-buy-full-speed` · 1 M capacity, 40 % of iterations attempt a purchase, no think time. The realistic mixed-traffic run: heavy browsing alongside buying, with cancellations and abandonment.
- **19:25** — `browse-and-buy-human-pace` · 100 K capacity, 5 % checkout share, 10–180 s think time. A behavioural run, not a capacity run.
- **19:46** — `buy-only-full-speed` · 1 M capacity, every iteration buys and pays, nobody cancels. The pure write-path stress test.

Only the first and third are capacity measurements, and only the third isolates the write path.

## Every verdict in the three reports is wrong

CORRECTION

**REPORTED: FAIL**↓ **should be****PASS**

## The human-pace run failed an invariant it could never satisfy

The `sellout: sold == totalCapacity` invariant is gated only on `CHECKOUT_PENDING_TIMEOUT_SECONDS ≤ 600`. Nothing checks whether the run actually ended by selling out. This run hit the 15-minute k6 ceiling instead (`stopReason: "k6-exited"`) and stopped at 99 788 of 100 000.

The missing tickets are accounted for exactly: 100 available, 112 still held in unexpired reservations, and the reaper handed those last 100 back to the pool *after* the load had already stopped, so no client remained to buy them. Every accounting invariant passed. This will misfire on any think-time profile.

### EVIDENCE

100 available + 99 788 tickets + 112 active = 100 000 capacity. Redis/Postgres drift 0. Drain complete, 0 pending.

REPORTED: PASS

↓ hides

SLO BREACH

## Both full-speed runs blew the latency threshold and were still marked pass

k6 declares `http_req_duration p(95) < 500ms`. The mixed run came in at **876 ms** and the buy-only run at **809 ms**. Both breached; both reports say “System result: pass”.

The verdict logic reads only correctness invariants. It never looks at `failedRate`, `duration.*` or the k6 exit code, and the renderer never prints them — so a threshold failure cannot reach the verdict or the page.

GENERATOR-SATURATED

↓ actually

3 CAUSES

## One recommendation fires for three unrelated reasons, none of them the generator

All three runs are stamped “do not claim the target RPS as backend capacity”. The load generator's hardware was never the limit in any of them:

- **Human-pace (52.6 % dropped)** — roughly 6 800 of 10 000 VUs sit asleep in think time (116 419 reservations × 60 s mean ÷ 1 024 s). The profile needs ~15 700 VUs. Backend median response was 0.59 ms; it was idle.
- **Buy-only (38.3 % dropped)** — offered 7 583 it/s × 1.306 s average iteration = **9 904 VUs against a 10 000 ceiling**. The pool was exhausted *by backend latency*. This is real back-pressure and the most valuable signal in the sweep, and it is the one being

discarded as an artefact.

- **Mixed (5.2 % dropped)** — only 3 896 VUs needed on average, but ~11 450 at the p95 iteration. Burst-driven, and it lands on the “invalid” side of the 5 % cutoff by 0.18 of a percentage point.

## What the system actually did

UNCORRECTED BY THE HARNESS

|                                                            |                                                      |                                                           |                                                      |
|------------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------|
| <b>5 300/s</b><br>Sustained orders,<br>buy-only plateau    | <b>0</b><br>5xx responses, all<br>three runs         | <b>0</b><br>Failed orders,<br>rollbacks,<br>compensations | <b>0</b><br>Redeliveries and<br>duplicate deliveries |
| <b>93.4 %</b><br>Orders persisted<br>within 1 s of payment | <b>15.0 s</b><br>Drain to zero<br>pending, every run |                                                           |                                                      |

The buy-only run sold **1 000 000 tickets in about 200 seconds of open sale**, holding a flat ~5 300 orders/s for two minutes with publish rate and consume rate tracking each other exactly. Nothing exceeded 2.5 s end-to-end, in any run.

The mixed run is the more interesting correctness result: it exercised every abandonment path at once — 91 055 cancellations and 45 651 expired reservations reclaimed by the reaper — and still reconciled to the ticket. Reserved 1 136 706 = paid 1 000 000 + cancelled 91 055 + expired 45 651, with Redis/Postgres drift of zero throughout.

Worth stating plainly: **every non-2xx response in all three runs was a deliberate business status** — 409 sold out, 425 sale not open, 410 payment expired. There was not one server error across roughly 14 million requests.

## Side by side

PHASE A

| MEASURE | BROWSE-AND-BUY<br>FULL-SPEED | BROWSE-AND-BUY<br>HUMAN-PACE | F |
|---------|------------------------------|------------------------------|---|
|---------|------------------------------|------------------------------|---|

| SHAPE OF THE RUN                            |                               |                 |                    |
|---------------------------------------------|-------------------------------|-----------------|--------------------|
| Seeded capacity                             | 1 000 000                     | 100 000         |                    |
| Phase-A duration                            | 748 s                         | 1 024 s         |                    |
| Ended because                               | <b>sold out</b>               | k6 ceiling      |                    |
| Offered /<br>executed<br>iterations per s   | 9 133 / 8 657                 | 9 076 / 4 305   | 7 58               |
| Dropped<br>iterations                       | <b>5.2 %</b>                  | <b>52.6 %</b>   |                    |
| VUs required vs<br>10 000 ceiling           | 3 896                         | <b>15 691</b>   |                    |
| THROUGHPUT                                  |                               |                 |                    |
| HTTP requests<br>per s                      | 10 115                        | 4 413           |                    |
| Order plateau<br>(completed per s)          | 3 050                         | ~90             |                    |
| Availability reads<br>per s                 | 5 190                         | 4 145           |                    |
| LATENCY                                     |                               |                 |                    |
| k6<br>http_req_duration<br>p95 (SLO 500 ms) | <b>876 ms</b>                 | <b>13.9 ms</b>  |                    |
| POST /buy p50 /<br>p95                      | 61 / 197 ms                   | 2.5 / 4.8 ms    | 92                 |
| POST /pay p50 /<br>p95 / p99                | <b>670 / 2 173 / 2 451 ms</b> | 13 / 24 / 46 ms | <b>398 / 936 /</b> |
| End-to-end mean<br>(publish →<br>persisted) | 218 ms                        | 24 ms           |                    |
| End-to-end p50 /                            |                               |                 |                    |

19.08.26, 06:50

Spike Sweep a05409f

|                                                                                                                                                      |                      |                      |             |
|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|----------------------|-------------|
| p95 / p99<br>(interpolated)                                                                                                                          | 0.18 / 0.07 / 0.77 s | 0.02 / 0.04 / 0.00 s | 0.03 / 1.30 |
| WRITE PATH                                                                                                                                           |                      |                      |             |
| buy_ticket                                                                                                                                           |                      |                      |             |
| mean (incl. pool wait)                                                                                                                               | 32.6 ms              | 1.5 ms               |             |
| DB pool queued acquirers, mean / peak                                                                                                                | 18 / 276             | 0 / 0                | 51          |
| CORRECTNESS                                                                                                                                          |                      |                      |             |
| Server errors (5xx)                                                                                                                                  | 0                    | 0                    |             |
| Failed orders / rollbacks / compensations                                                                                                            | 0                    | 0                    |             |
| Redis ↔ Postgres drift                                                                                                                               | 0                    | 0                    |             |
| Client-side connection failures                                                                                                                      | 61 377 (0.80 %)      | 4                    | 35 481      |
| Percentiles marked “interpolated” are computed across histogram buckets; the reports quote bucket edges in which is why they read 1.000 s / 2.500 s. |                      |                      |             |

Reporting defects

FIX BEFORE THE NEXT SWEEP

transport\_errors counts business responses, not transport errors

DEFECT

The counter increments whenever k6 populates `res.error_code`, which happens for *any* non-2xx status. For `/buy` that means every sold-out 409 and every too-early 425 is filed as an infrastructure error.

The mixed run's headline “1 492 585 transport errors” decomposes into 1 425 420 sold-

out responses, 29 775 too-early responses and 61 377 genuine connection failures. The cool-down phase of the buy-only run reports 60 000 transport errors against 60 000 passing checks and an `http_req_failed` rate of zero — a 100 % false-positive rate.

The README documents the opposite behaviour, describing it as requests that got no application response at all. The `error_code` dimension that would separate the two cases is deliberately dropped from the export to limit metric cardinality.

#### SOURCE

`load-tests/lib/scenario-helpers.js:200-205` — single increment site, called unconditionally after every request. Documented contrary at `load-tests/README.md:124-126`.

## The strongest run got the weakest verification

DEFECT

The sellout, reaper and expiry invariants are only added when

`CHECKOUT_PENDING_TIMEOUT_SECONDS ≤ 600`. The buy-only profile sets it to 900, so the run that actually sold a million tickets was checked against five base invariants and never against sellout at all — while the human-pace run, which was never going to sell out, was checked and failed.

The gate is backwards on both ends: it should key on whether the run reached sellout, not on a timeout value.

#### SOURCE

`scripts/load-test/lib/derive.mjs:301-325` — gate is `expiryExercised`, with no reference to `stopReason` or `availableAtStop`.

## The event-loop lag metric never observed the load

DEFECT

Before the buy-only run the API reported mean 10.6 ms and max 11.5 ms. After 260 seconds at 8 500 requests per second it reported mean 11.0 ms and max 12.1 ms, with min pinned at 10.0 ms. In every service and every run, max equals p99 exactly.

Two causes, both confirmed in `prom-client@15.1.3`: the monitor runs at 10 ms resolution, so the sampling period *is* the floor being reported; and the histogram is reset inside the collect callback, so each scrape only ever describes the last five seconds. The Grafana panel built on it is decorative.

#### SOURCE

`prom-client/lib/metrics/eventLoopLag.js:63` — `histogram.reset()` on every collect. Default precision 10 ms at `lib/defaultMetrics.js:44`; never overridden in either service.

## Histogram buckets are blind exactly where the answers live

OBSERVATION

Both `http_request_duration_seconds` and `order_e2e_latency_seconds` step 0.5 → 1 → 2.5 → 5 s. In the buy-only run, 6.6 % of orders fall in the 1–2.5 s bucket and *both* p95 and p99 land inside it, so the report's “p95 / p99 = 2.500 s / 2.500 s” only means “somewhere between 1 and 2.5 seconds”. Interpolating gives 1.36 s and 2.27 s.

## Performance findings

REAL, IN THE SYSTEM

### `POST /pay` is the slowest thing users touch, by two orders of magnitude

PRIORITY

Every other endpoint is single- or double-digit milliseconds. The publish path is not: 670 ms p50 and 2.45 s p99 in the mixed run, 398 ms p50 and 1.55 s p99 in the buy-only run. This is what breaks the p95 SLO on both.

The Grafana panel shows p50 climbing monotonically to ~950 ms across a period when order throughput was completely flat, with a p99 spike to 4.93 s at the tail — a queue building inside the publisher rather than a slow dependency.

It also compounds with the reported latency rather than overlapping it. The report's end-to-end figure starts at publish, so a mixed-run buyer waits 670 ms in `/pay` and *then* the 180 ms median to persistence.

### The worker's 50-connection pool is the write-path queue, and it has no timeout

PRIORITY

At the buy-only plateau the pool sat pinned at its 50-connection maximum with **3 760 queued acquirers** at peak. The `buy_ticket` timer wraps `pg.Pool.query()`, which acquires internally, so its 172.9 ms mean is mostly queueing: roughly 15 ms of work on a connection and ~158 ms waiting for one. The uncontended reference from the human-pace run is 1.5 ms.

Two structural notes. `connectionTimeoutMillis` is unset, so queued acquirers wait indefinitely — 3 760 deep, that is an availability risk with no fail-fast. And

`PUBSUB FLOW CONTROL MAX MESSAGES=500` is a soft bound only: the client defaults to

`allowExcessMessages: true`, so in-flight handlers can exceed it. The pool is the real limiter.

## The hot-row panel cannot see the contention this schema would produce

BLIND SPOT

`db_locks_waiting` read zero throughout, which looks like a clean bill of health. It isn't one. The gauge filters on `wait_event_type = 'Lock'`, but every `buy_ticket` call takes `FOR KEY SHARE` twice on the same `events` tuple via foreign keys. Those locks are mutually compatible, so contention surfaces as MultiXact SLRU pressure under `LWLock` — a wait class this query excludes by construction.

With roughly 3 400 writes per second converging on one of two rows in `events`, zero is not evidence of absence.

## Two latent write-path hazards found while tracing

OBSERVATION

`tickets` has no index on `order_id`. The duplicate-delivery branch of `buy_ticket` looks up by that column, so each redelivery sequentially scans a 1 M-row, 103 MB table — around 100 ms of CPU while holding a pool slot. These runs saw no redeliveries, so it never bit; under retry pressure it would.

**A dead `buy_ticket` overload still exists in the database.** The legacy three-argument version, which still contains the `UPDATE events SET sold_count = sold_count + 1` hot-row write that migration 0009 was written to remove, was never dropped. The worker calls the four-argument version, so the measured path is clean — but the old one is one wrong call site away from returning.

## Connection establishment stalls under load

OBSERVATION

Both full-speed runs show `http_req_connecting` peaking near 15 s — the shape of an overflowing accept queue — against 3.0 s in the idle human-pace run. Cross-checking k6's request count against the API's own counters, roughly 60 800 requests in the mixed run and 34 300 in the buy-only run never reached the service at all, matching their client-side failure counts almost exactly.

Both services are single unclustered Node processes. Tempting as it is to call the event loop saturated, the artifacts cannot support it: `process_cpu_seconds_total` covers the whole process including the libuv threadpool, and the metric that would settle it is the broken lag histogram above.



# What I would change

ORDERED BY PAYOFF

## 01 Rename and split `transport_errors` .

Separate  
genuine  
status-  
0  
failures  
from  
expected  
business  
statuses,  
or  
the  
next  
reader  
will  
chase  
1.5  
million  
phantom  
errors.  
Fix  
the  
README  
to  
match  
whichever  
behaviour  
you  
keep.

## 02 Gate the sellout invariant on `stopReason` , not on a timeout value.

That  
removes  
the  
false  
failure

on  
think-  
time  
profiles  
and  
restores  
real  
verification  
to  
the  
buy-  
only  
profile,  
which  
currently  
has  
none.

### 03 Feed k6 thresholds into the system verdict

, and `failedRate`  
print

, the  
duration  
percentiles  
and  
the  
phase  
exit  
code  
in  
the  
report.  
Two  
SLO  
breaches  
passed  
silently  
through  
this  
sweep.

### 04 Profile the `POST /pay` publish path.

It is

the  
dominant  
user-  
facing  
latency,  
it is  
the  
sole  
cause  
of  
both  
SLO  
breaches,  
and  
its  
median  
climbs  
under  
flat  
throughput.

**05**    **Set** `connectionTimeoutMillis` on the worker pool

and  
decide  
deliberately  
whether  
50 is  
the  
right  
ceiling.  
Unbounded  
waiting  
behind  
a 3  
760-  
deep  
queue